TrafficAI — Parking Occupancy Detection Feature

This document explains how the parking occupancy detection feature works end-to-end: the detection logic, the API contract (inputs and outputs), and the coordinate-picking tool used to set up a new parking lot.

---

1. Feature Overview

The parking feature answers one question for a given camera snapshot of a parking lot: **which marked parking spaces are occupied, and which are vacant?**

It works by combining two pieces of information:

- A **photo of the parking lot** (uploaded by the user/client).
- A **list of parking slot boundaries** — the pixel coordinates that outline each individual parking space in that photo (defined once, ahead of time, using the coordinate-picking tool).

The system detects vehicles in the photo using a YOLO object-detection model, then checks which detected vehicles fall inside which slot boundary. A slot is marked **Occupied** if a vehicle's center point lands inside its boundary, and **Vacant** otherwise. The result is returned as structured data, along with a copy of the original photo with **green boxes drawn over occupied slots and red boxes over vacant slots**, so the result can be visually verified at a glance.

---

2. How the Detection Logic Works (`predictors/parking.py`)

This is the core processing module behind the `/predict/parking` endpoint. Its job, step by step:

Step 1 — Decode the uploaded image correctly. Phone and camera photos are often stored with hidden rotation metadata (EXIF orientation tags). If this isn't accounted for, the image can be read "sideways," which throws off every slot coordinate. The module decodes the image in a way that **always corrects for this rotation first**, so the pixel layout matches exactly what a person sees when viewing the photo normally.

Step 2 — Run vehicle detection. The corrected image is passed through a YOLO object-detection model, which scans the entire photo and returns a list of every detected vehicle (cars, motorcycles, buses, trucks), each with a bounding box and a confidence score.

Step 3 — Reduce each vehicle to a single point. For every detected vehicle, the module calculates the **center point** of its bounding box. This center point is what gets checked against each parking slot boundary — a simpler and more reliable check than comparing two overlapping rectangles.

Step 4 — Check each slot for occupancy. For every parking slot boundary provided in the request, the module checks whether **any** vehicle's center point falls inside that slot's polygon shape. If yes, the slot is marked **Occupied**; if no vehicle center falls inside it, the slot is marked **Vacant**.

Step 5 — Calculate summary statistics. Once every slot has a status, the module calculates:

- Total number of slots
- Number of occupied slots
- Number of vacant slots
- Occupancy rate (as a percentage)
- An average confidence score across all detected vehicles

Step 6 — Draw the annotated image. A copy of the original (corrected) photo is annotated: each slot boundary is outlined and lightly filled — **green for Occupied, red for Vacant** — with a small label showing the slot ID and its status. This gives a clear visual confirmation alongside the raw numbers.

Step 7 — Encode the annotated image. The annotated image is encoded into a Base64 text string (a way of representing image data as plain text), so it can be included directly inside a JSON API response rather than as a separate file.

Step 8 — Return the full result. All of the above — the summary statistics, the per-slot status list, and the Base64 annotated image — are packaged together and returned as the API response.

---

3. API Endpoint: `/predict/parking`

Method: POST Content type: multipart/form-data (because an image file is being uploaded alongside text fields)

Input Fields

Field	Type	Required	Description
parking_image	File	**Yes**	The photo of the parking lot to analyze. Must be an image file.

Field	Type	Required	Description
parking_id	Text	**Yes**	An identifier for which parking lot/camera this photo belongs to (e.g. `"lot-1"`).
parking_slots	Text (JSON string)	**Yes**	A JSON-encoded list describing every parking slot's boundary in this photo. Each entry has an **id** (slot number) and **coordinates** (a flat list of alternating x, y pixel points outlining the slot's corners).

Important constraint: the **parking_slots** coordinates must correspond to the *exact same image resolution and orientation* as the **parking_image** being uploaded. If the slot boundaries were defined on a different photo, a resized photo, or a photo with different rotation, the results will be inaccurate or misaligned.

Output Fields

Field	Type	Description
total_slots	Number	Total number of parking slots evaluated.
occupied_slots	Number	Count of slots currently occupied by a vehicle.
vacant_slots	Number	Count of slots currently empty.
occupancy_rate	Number	Percentage of slots occupied (0–100).
confidence_score	Number	Average detection confidence across all vehicles found in the image.
slot_status	List	Per-slot breakdown — each entry has the slot's **id** and its **status** (**"Occupied"** or **"Vacant"**).
annotated_image	Text (Base64)	The parking lot photo with colored boxes drawn over every slot — green for occupied, red for vacant — encoded as a Base64 string. Can be decoded back into a viewable image (e.g. a `.jpg` file).

Example Response

{ "total_slots": 12, "occupied_slots": 5, "vacant_slots": 7, "occupancy_rate": 42, "confidence_score": 0.87, "slot_status": [{ "id": 1, "status": "Occupied" }, { "id": 2, "status": "Vacant" }, { "id": 3, "status": "Occupied" }], "annotated_image": "/9j/4AAQSkZJRgABAQAAAQABAAD... (long Base64 string)" }

Error Responses

Situation	Response
Uploaded file is not an image	**400 Bad Request** — "Uploaded file must be an image."
Uploaded image file is empty	**400 Bad Request** — "Empty image file."
parking_slots isn't valid JSON	**400 Bad Request** — explains the expected JSON format.
parking_slots is an empty list	**400 Bad Request** — "parking_slots list cannot be empty."
A slot's coordinate list is malformed (odd number of values, or fewer than 3 points)	**400 Bad Request** — explains the coordinate format requirement.
The detection model file is missing on the server	**500 Internal Server Error** — points to the missing model path.

---

4. The Slot Coordinate-Picking Tool

Before the API can evaluate a parking lot photo, someone has to define **where each parking slot is** in that photo — these are the **parking_slots** coordinates sent in every request. Since manually guessing pixel coordinates is impractical, a small standalone tool was built for this purpose.

What it does

The tool opens a chosen parking lot photo in an interactive window. The person running it simply **clicks the corners of each parking space** directly on the image (usually four clicks per slot, tracing around its boundary), then presses a key to lock that slot in and move to the next one. Once every slot has been marked, the tool saves all of the coordinates into a ready-to-use JSON file — and also prints them in a single-line format that can be pasted directly into the API request.

Why this matters for accuracy

The tool decodes the image using the **exact same orientation-correction logic** as the API itself. This guarantees that whatever pixel position is clicked on screen lines up precisely with how the API will interpret that same photo — eliminating the earlier issue where slot boundaries appeared shifted because of differing image orientation handling.

The tool also automatically scales large photos down to fit comfortably on screen for easier clicking, while still recording and saving the coordinates at the photo's **true, full resolution** — so the saved coordinates always remain accurate regardless of how zoomed-in or zoomed-out the on-screen view was.

Output of the tool

A JSON list of slot definitions, where each slot has:

- An **id** (a sequential slot number, assigned automatically in the order the slots were marked)
- A **coordinates** list — the flat sequence of x, y pixel points outlining that slot's shape

This output is exactly the format the **parking_slots** field expects in the API request — no conversion needed.

---

5. End-to-End Workflow Summary

1. **Capture or select** a parking lot photo from the camera/source you intend to monitor.
2. **Run the coordinate-picking tool** on that exact photo, clicking the corners of every visible parking slot.
3. **Copy the resulting JSON** of slot coordinates from the tool's output.
4. **Send a request** to the `/predict/parking` endpoint with: the same photo as **parking_image**, a chosen **parking_id**, and the copied JSON as **parking_slots**.
5. **Receive the response**, containing the occupancy summary, per-slot statuses, and the Base64 annotated image.
6. **Decode the annotated_image** field (or view it via a Base64-to-image viewer) to visually confirm the green/red slot markings match reality.
7. For a **fixed camera setup**, steps 2–3 only need to be done once — the same slot coordinates can be reused for every new photo taken from that same camera angle, since the parking spaces don't move.

Note: Slot coordinates are tied to a specific camera position and image resolution. If the camera is moved, repositioned, or the image resolution changes, the coordinate-picking step must be repeated for accurate results.